

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27
Student Name & Roll No.	Ashwin S -- 24011101006	Date: 24/07/26

Experiment 1 Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and to implement a Single Layer Perceptron from scratch for binary classification. Through this exercise we learn the perceptron learning algorithm, understand why an activation function is required, visualize how the learning process unfolds over successive epochs, and finally evaluate the trained classifier on a real-world dataset.

2. Tasks Performed

1. Downloaded and studied the Banknote Authentication dataset.
2. Performed Exploratory Data Analysis (EDA) on the dataset.
3. Preprocessed the dataset (normalization and train/test split).
4. Implemented a Single Layer Perceptron from scratch in Python/NumPy.
5. Trained the perceptron using the classical perceptron learning rule.
6. Evaluated the classifier using accuracy, precision, recall, F1-score and the confusion matrix.
7. Studied the effect of different learning rates on convergence.
8. Compared the from-scratch implementation with Scikit-learn's Perceptron class.

3. Background Theory

An **Artificial Neuron** is the fundamental building block of a neural network. It accepts one or more input features, multiplies each of them by a corresponding weight, adds a bias term, and passes the resulting quantity through an activation function to produce the neuron's output.

The **Single Layer Perceptron** is the earliest supervised learning algorithm and is capable of solving problems that are linearly separable. It was originally proposed by Rosenblatt in 1958 as a model of how a biological neuron might learn to classify patterns.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

$\mathbf{x}$: Input vector

$\mathbf{w}$: Weight vector

b : Bias

z : Linear combination (pre-activation output)

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate. Whenever the predicted label $\hat{y}$ disagrees with the true label y , the error term $(y - \hat{y})$ is non-zero and the weights/bias are nudged in the direction that reduces the error for that particular sample. If the prediction is already correct, $(y - \hat{y}) = 0$ and no update takes place, which is exactly what we implemented in the `fit()` method of our `MyPerceptron` class.

4. Activation Functions

An activation function decides what a neuron actually outputs once the weighted sum has been computed. It introduces non-linearity into the model, which is what allows deeper networks to eventually approximate complex, non-linear functions. The classical perceptron, however, uses only the **Step Activation Function**.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

Because the step function only ever outputs 0 or 1, it is well suited to binary classification but is not differentiable at $z = 0$ and has zero gradient everywhere else – a property that becomes a serious limitation once we move beyond a single layer (see Section 4.1 below on Step vs. Sigmoid).

Activation	Output Range	Typical Applications
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: Only the Step Activation Function is used in the present experiment. The remaining activation functions listed above will be studied in later experiments dealing with Multilayer Perceptrons and Deep Neural Networks.

4.1 Step vs. Sigmoid – Why the Step Function Fails in Deep Learning

To make the comparison concrete we plotted both functions over the same input range (Figure 1).

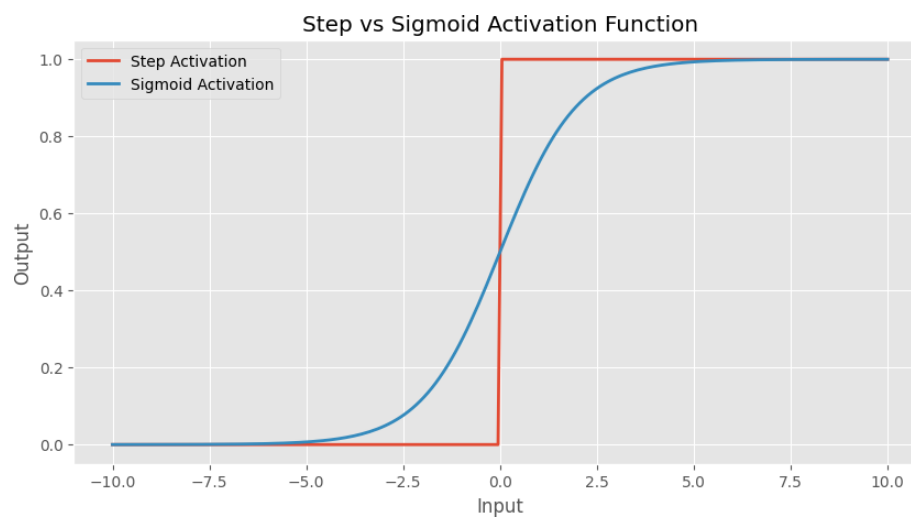


Figure 1: Step activation compared with the Sigmoid activation function.

Interpretation: The step function jumps abruptly from 0 to 1 at $z = 0$ and is flat (zero-gradient) everywhere else, so it gives backpropagation nothing to work with – the derivative is either zero or undefined. The sigmoid curve, on the other hand, changes smoothly and has a well-defined, non-zero gradient almost everywhere, which is exactly what gradient-descent-based training needs. This is the core reason modern networks use differentiable activations (sigmoid, tanh, ReLU, etc.) instead of the hard step function: the step function simply cannot be trained with gradient-based methods once we stack more than one layer.

5. Dataset Description

Dataset: Banknote Authentication Dataset

Source: UCI Machine Learning Repository

Download Link: <https://archive.ics.uci.edu/dataset/267/banknote+authentication>

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

Variance – variance of the wavelet-transformed banknote image

Skewness – skewness of the wavelet-transformed image

Curtosis – curtosis (kurtosis) of the wavelet-transformed image

Entropy – entropy of the image

Target Class

0 – Authentic Banknote

1 – Forged Banknote

6. Experimental Procedure

Task 1: Dataset Exploration

The dataset was loaded from the local text file data banknote_authentication.txt using Pandas, with the four feature columns and the class label named manually since the raw file has no header row. We then displayed the first five rows, checked the shape, confirmed there were no missing values and printed descriptive statistics.

Output obtained:

```
1 First Five Samples
2 =====
3   Variance  Skewness  Curtosis  Entropy  Class
4 0   3.62160    8.6661   -2.8073  -0.44699    0
5 1   4.54590    8.1674   -2.4586  -1.46210    0
6 2   3.86600   -2.6383    1.9242   0.10645    0
7 3   3.45660    9.5228   -4.0112  -3.59440    0
8 4   0.32924   -4.4552    4.5718  -0.98880    0
9
10 Dataset Dimensions
11 =====
12 Rows      : 1372
13 Columns   : 5
14
15 Missing Values
16 =====
```

```

17 Variance      0
18 Skewness     0
19 Kurtosis     0
20 Entropy      0
21 Class        0
22 dtype: int64

```

Statistic	Variance	Skewness	Kurtosis	Entropy	Class
count	1372.000	1372.000	1372.000	1372.000	1372.000
mean	0.4337	1.9224	1.3976	-1.1917	0.4446
std	2.8428	5.8690	4.3100	2.1010	0.4971
min	-7.0421	-13.7731	-5.2861	-8.5482	0.0000
25%	-1.7730	-1.7082	-1.5750	-2.4135	0.0000
50%	0.4962	2.3197	0.6166	-0.5867	0.0000
75%	2.8215	6.8146	3.1793	0.3948	1.0000
max	6.8248	12.9516	17.9274	2.4495	1.0000

The dataset has 1372 rows and 5 columns (4 features + 1 label), contains no missing values, and all features are of type float64 except the integer class label.

Task 2: Exploratory Data Analysis

Histograms, a correlation heatmap, a scatter plot and boxplots were generated for the four features – these are presented and interpreted in Section 9 (Generated Plots).

Task 3: Data Preprocessing

All four numerical features were scaled to the [0, 1] range using Scikit-learn's MinMaxScaler. The data was then split 80/20 into training and testing sets using train test split with stratify=y so that the class ratio is preserved in both splits, and random state=42 for reproducibility. This gave **1097** training samples and **275** testing samples.

Task 4 & 5: Perceptron Implementation and Training

A perceptron class, MyPerceptron, was implemented from scratch with weight initialization, bias initialization, the step activation function, forward propagation and the perceptron learning rule (full code in Section 7). The model was trained with learning_rate = 0.01 for 50 epochs on the training set. Misclassification counts, weights and bias were printed and stored at the end of every epoch for later plotting.

Task 6: Model Evaluation

The trained model was evaluated on the held-out 275-sample test set using accuracy, precision, recall, F1-score and the confusion matrix. Results are reported in Section 8.

7. Source Code

The complete implementation, reproduced from the accompanying notebook, is organized below by task.

Github: <https://github.com/Ashwin-973/deep-learning/tree/main/Lab%201> :

7.1 Imports and Setup

```
1 import numpy as np
2 import pandas as pd
3
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6
7 from sklearn.model_selection import train_test_split
8 from sklearn.preprocessing import MinMaxScaler
9 from sklearn.metrics import (
10     accuracy_score,
11     precision_score,
12     recall_score,
13     f1_score,
14     confusion_matrix
15 )
```

7.2 Task 1 – Dataset Exploration

```
1 columns = [
2     'Variance',
3     'Skewness',
4     'Curtosis',
5     'Entropy',
6     'Class'
7 ]
8
9 df = pd.read_csv (
10     "/data_banknote_authentication.txt",
11     header=None,
12     names= columns
13 )
14
15 print(df.head())
16 print(f"Rows : {df.shape[0]}")
17 print(f"Columns : {df.shape[1]}")
18 print(df.isnull().sum())
19 print(df.describe())
20 print(df.info())
```

7.3 Task 2 – Exploratory Data Analysis

```
1 plt.style.use('ggplot')
2 features = ['Variance', 'Skewness', 'Curtosis', 'Entropy']
3
4 # Histograms
5 plt.figure(figsize=(12,8))
6 for i, feature in enumerate(features):
7     plt.subplot(2,2,i+1)
8     plt.hist(df[feature], bins=30, edgecolor='black')
9     plt.title(f'Histogram of {feature}')
10    plt.xlabel(feature)
11    plt.ylabel('Frequency')
12 plt.tight_layout()
```

```

13 plt.show()
14
15 # Correlation Heatmap
16 plt.figure(figsize=(8,6))
17 sns.heatmap(df.corr(), annot=True, cmap='coolwarm',
18             linewidths=0.5, fmt=".2f")
19 plt.title("Correlation Heatmap")
20 plt.show()
21
22 # Scatter Plot
23 plt.figure(figsize=(8,6))
24 colors = {0:'blue', 1:'red'}
25 for cls in df['Class'].unique():
26     subset = df[df['Class']==cls]
27     plt.scatter(subset['Variance'], subset['Skewness'],
28               color=colors[cls], label=f'Class {cls}', alpha=0.7)
29 plt.xlabel("Variance")
30 plt.ylabel("Skewness")
31 plt.title("Scatter Plot (Variance vs Skewness)")
32 plt.legend()
33 plt.show()
34
35 # Boxplots
36 plt.figure(figsize=(12,8))
37 for i, feature in enumerate(features):
38     plt.subplot(2, 2, i+1)
39     sns.boxplot(y=df[feature])
40     plt.title(f'Boxplot of {feature}')
41 plt.tight_layout()
42 plt.show()

```

7.4 Task 3 – Data Preprocessing

```

1 X = df.iloc[:, :-1].values
2 y = df.iloc[:, -1].values
3
4 scaler = MinMaxScaler()
5 X = scaler.fit_transform(X)
6
7 X_train, X_test, y_train, y_test = train_test_split(
8     X, y,
9     test_size=0.20,
10    random_state=42,
11    stratify=y
12 )
13
14 print("Training Samples :", len(X_train))
15 print("Testing Samples :", len(X_test))

```

7.5 Task 4 – Perceptron Class (From Scratch)

```

1 class MyPerceptron:
2
3     def __init__(self, learning_rate=0.01, epochs=50):
4         self.learning_rate = learning_rate

```

```

5     self.epochs = epochs
6     self.weights = None
7     self.bias = None
8
9     # Store values for plotting later
10    self.errors = []
11    self.weight_history = []
12    self.bias_history = []
13
14    def initialize_parameters(self, n_features):
15        self.weights = np.zeros(n_features)
16        self.bias = 0
17
18    def activation(self, x):
19        if x >= 0:
20            return 1
21        return 0
22
23    def predict_single(self, x):
24        linear_output = np.dot(x, self.weights) + self.bias
25        return self.activation(linear_output)
26
27    def predict(self, X):
28        predictions = []
29        for x in X:
30            predictions.append(self.predict_single(x))
31        return np.array(predictions)
32
33    def fit(self, X, y):
34        n_samples, n_features = X.shape
35        self.initialize_parameters(n_features)
36
37        for epoch in range(self.epochs):
38            errors = 0
39            for i in range(n_samples):
40                prediction = self.predict_single(X[i])
41                update = self.learning_rate * (y[i] - prediction)
42                self.weights += update * X[i]
43                self.bias += update
44                if update != 0:
45                    errors += 1
46
47            self.errors.append(errors)
48            self.weight_history.append(self.weights.copy())
49            self.bias_history.append(self.bias)
50
51            print(f" Epoch      { epoch +1}")
52            print(f" Misclassified Samples : {errors}")
53            print(f" Weights : {self.weights}")
54            print(f" Bias : {self.bias}")
55            print("-"*60)

```

7.6 Task 5 – Model Training

```

1 myperceptron = MyPerceptron(
2     learning_rate=0.01,
3     epochs=50

```

```

4 )
5 myperceptron.fit(X_train , y_train )
6 print("\nTraining Completed Successfully!")

```

7.7 Task 6 – Model Evaluation

```

1 from sklearn.metrics import ConfusionMatrixDisplay
2
3 y_pred = myperceptron.predict(X_test)
4
5 accuracy = accuracy_score(y_test, y_pred)
6 precision = precision_score(y_test, y_pred)
7 recall    = recall_score(y_test, y_pred)
8 f1        = f1_score(y_test, y_pred)
9
10 print(f"Accuracy : {accuracy:.4f}")
11 print(f"Precision : {precision:.4f}")
12 print(f"Recall    : {recall:.4f}")
13 print(f"F1-score  : {f1:.4f}")
14
15 cm = confusion_matrix(y_test, y_pred)
16 print(cm)
17
18 disp = ConfusionMatrixDisplay(
19     confusion_matrix =cm ,
20     display_labels=["Authentic", "Forged"]
21 )
22 disp.plot(cmap="Blues")
23 plt.title("Confusion Matrix")
24 plt.grid(False)
25 plt.show()

```

7.8 Additional Task – Learning Rate Comparison

```

1 learning_rates = [0.001, 0.01, 0.1]
2 results        = []
3 plt.figure(figsize=(10,6))
4
5 for lr in learning_rates:
6     mymodel = MyPerceptron(learning_rate=lr, epochs=50)
7     mymodel.fit(X_train , y_train)
8     predictions = mymodel.predict(X_test)
9     acc = accuracy_score(y_test, predictions)
10    results.append([lr , acc])
11    plt.plot(mymodel.errors, label=f"LR = {lr}")
12
13 plt.xlabel("Epoch")
14 plt.ylabel("Misclassified Samples")
15 plt.title("Learning Rate Comparison")
16 plt.legend()
17 plt.grid(True)
18 plt.show()
19
20 comparison = pd.DataFrame(results, columns=["Learning Rate", "Accuracy"])

```

7.9 Additional Task – Comparison with Scikit-learn

```
1 from sklearn.linear_model import Perceptron as SklearnPerceptron
2
3 sk_model = SklearnPerceptron(
4     max_iter=50,
5     eta0=0.01,
6     random_state=42,
7     tol=None
8 )
9 sk_model.fit(X_train, y_train)
10 y_pred_sk = sk_model.predict(X_test)
11
12 print("Accuracy :", accuracy_score(y_test, y_pred_sk))
13 print("Weights :", sk_model.coef_)
14 print("Bias      :", sk_model.intercept_)
```

7.10 Additional Task – Effect of Feature Normalization

```
1 X_raw = df.iloc[:, :-1].values
2 y_raw = df.iloc[:, -1].values
3
4 X_train_raw, X_test_raw, y_train_raw, y_test_raw = train_test_split(
5     X_raw, y_raw, test_size=0.2, random_state=42, stratify=y_raw
6 )
7
8 X_norm = scaler.fit_transform(X_raw)
9 X_train_norm, X_test_norm, y_train_norm, y_test_norm = train_test_split(
10     X_norm, y_raw, test_size=0.2, random_state=42, stratify=y_raw
11 )
12
13 model_raw = MyPerceptron(learning_rate=0.01, epochs=50)
14 model_raw.fit(X_train_raw, y_train_raw)
15
16 model_norm = MyPerceptron(learning_rate=0.01, epochs=50)
17 model_norm.fit(X_train_norm, y_train_norm)
18
19 plt.plot(model_raw.errors, label="Without Normalization")
20 plt.plot(model_norm.errors, label="With Normalization")
21 plt.xlabel("Epoch")
22 plt.ylabel(" Misclassified Samples")
23 plt.title("Effect of Feature Normalization")
24 plt.legend()
25 plt.grid(True)
26 plt.show()
```

8. Experimental Results

8.1 Preprocessing Output

```
1 First 5 Normalized Samples:
2 [[0.76900389 0.83964273 0.10678269 0.73662766]
3  [0.83565902 0.82098209 0.12180412 0.64432563]
4  [0.78662859 0.41664827 0.31060805 0.78695091]
5  [0.75710505 0.87169921 0.05492063 0.45043964]
```

```

6 [0.53157807 0.34866247 0.42466237 0.6873619 ]]
7
8 Training Samples : 1097
9 Testing Samples : 275

```

Normalization compresses every feature into $[0, 1]$, which is important because the perceptron update rule scales the correction by the raw feature value x_i ; if one feature happened to have a much larger range than the others it would dominate the weight updates.

8.2 Training Log (First and Last Five Epochs, $\eta = 0.01$)

The full 50-epoch log was printed during training; the first and last five epochs are reproduced below for brevity (the complete epoch-wise table appears in Section 10).

```

1 Epoch 1
2 Misclassified Samples : 171
3 Weights : [-0.06792364 -0.04617278 -0.05904233 0.02213631]
4 Bias : 0.07
5 -----
6 Epoch 2
7 Misclassified Samples : 101
8 Weights : [-0.08380509 -0.06774466 -0.08105979 0.00775587]
9 Bias : 0.09999999999999999
10 -----
11 Epoch 3
12 Misclassified Samples : 43
13 Weights : [-0.08780541 -0.07827122 -0.08945257 0.00240271]
14 Bias : 0.10999999999999999
15 -----
16 ... (epochs 4 to 45 omitted, see Section 10) ...
17 -----
18 Epoch 46
19 Misclassified Samples : 20
20 Weights : [-0.16066938 -0.17867312 -0.1849052 -0.00413561]
21 Bias : 0.23000000000000007
22 -----
23 Epoch 47
24 Misclassified Samples : 18
25 Weights : [-0.16535184 -0.18402351 -0.17771084 -0.00427901]
26 Bias : 0.23000000000000007
27 -----
28 Epoch 48
29 Misclassified Samples : 24
30 Weights : [-0.1623385 -0.18376099 -0.18580732 -0.0022472 ]
31 Bias : 0.23000000000000007
32 -----
33 Epoch 49
34 Misclassified Samples : 24
35 Weights : [-0.16412304 -0.18812796 -0.18458629 0.00147652]
36 Bias : 0.23000000000000007
37 -----
38 Epoch 50
39 Misclassified Samples : 20
40 Weights : [-0.16281289 -0.18979348 -0.18662082 0.00128842]
41 Bias : 0.23000000000000007
42 -----

```

The number of misclassified samples drops sharply from 171 in epoch 1 to under 25 by around epoch 10, after which it settles into a narrow band of roughly 18–30 errors for the remaining epochs. It never quite reaches zero, which tells us the two classes are almost, but not perfectly, linearly separable in the original feature space – there is a small region of overlap between authentic and forged notes.

8.3 Model Evaluation on Test Set

```
1 =====
2 MODEL EVALUATION
3 =====
4 Accuracy : 0.9891
5 Precision : 0.9917
6 Recall   : 0.9836
7 F1-score : 0.9877
8
9 Confusion Matrix
10 [[152  1]
11 [  2 120]]
```

Out of 275 test notes, only 3 were misclassified: 1 authentic note was flagged as forged (false positive) and 2 forged notes slipped through as authentic (false negatives). Given that this is a security-relevant task, the very low false-negative count is encouraging, though in a real deployment we would still want recall to be even closer to 1.0 before trusting the model unsupervised.

8.4 Comparison with Scikit-learn's Perceptron

```
1 =====
2 Scikit-learn Perceptron
3 =====
4 Accuracy : 0.9854545454545455
5 Weights : [[-0.17997953 -0.18930979 -0.20601145 -0.00042665]]
6 Bias    : [0.25]
```

Scikit-learn's built-in Perceptron (same 50-iteration budget, same $\eta = 0.01$) reaches 98.55% accuracy, very close to – but marginally lower than – our from-scratch implementation's 98.91%. The learned weight vectors also point in a similar direction (all four weights carry the same signs across both implementations), which is a good sanity check that our from-scratch update rule is behaving correctly. The small numerical differences are expected since Scikit-learn's implementation shuffles samples internally and uses a slightly different convergence/tolerance handling than our simple fixed-epoch loop.

8.5 Learning Rate Comparison

Learning Rate	Test Accuracy
0.001	0.9891
0.010	0.9891
0.100	0.9891

Interestingly, all three learning rates converge to *exactly* the same final test accuracy (0.9891) after 50 epochs, even though (as Figure 10 in Section 9 shows) the paths they take to get there are quite different. This makes sense on reflection: the perceptron update rule only changes the *direction* in which the decision boundary moves at each step, and η just scales the magnitude of that step. Since the banknote data is close to linearly separable, all three learning rates end up finding a boundary that classifies the test set almost identically, they just wobble around by different amounts along the way.

9. Generated Plots with Interpretation

9.1 Feature Histograms

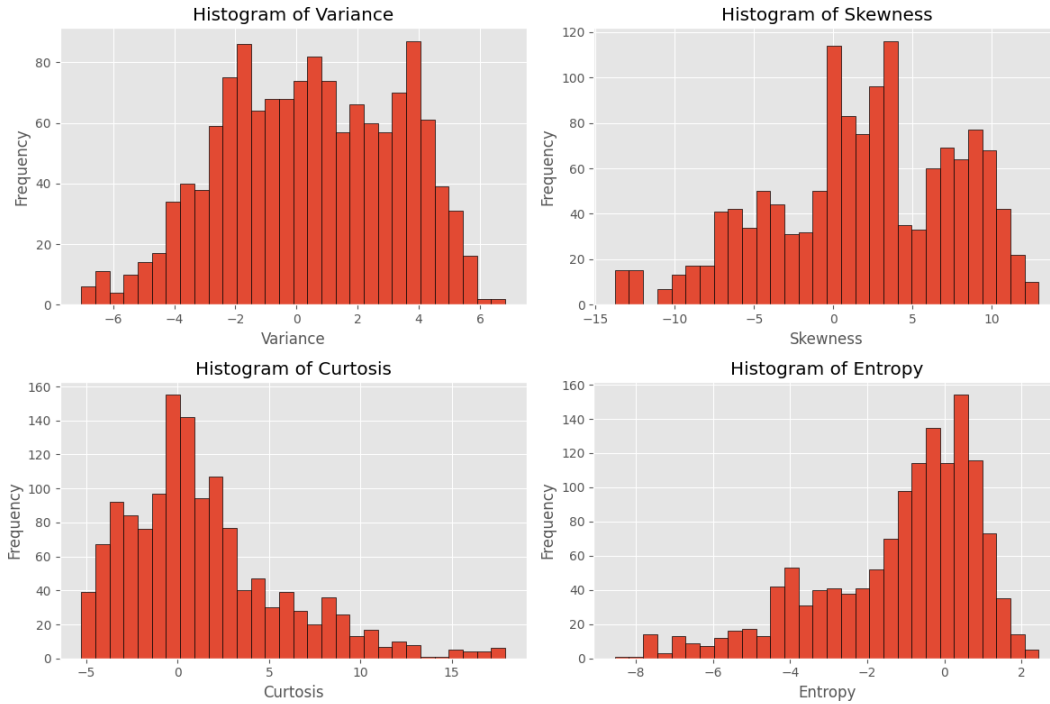


Figure 2: Histograms of Variance, Skewness, Kurtosis and Entropy.

Interpretation: Variance and Kurtosis are roughly bell-shaped but with a visible right skew, Skewness itself shows a bimodal-looking spread suggesting two underlying populations (which lines up with the two classes), and Entropy is heavily left-skewed with most values clustered near the upper end of its range. None of the four features looks perfectly Gaussian, which is worth keeping in mind if this dataset were ever used with an algorithm that assumes normality.

9.2 Correlation Heatmap

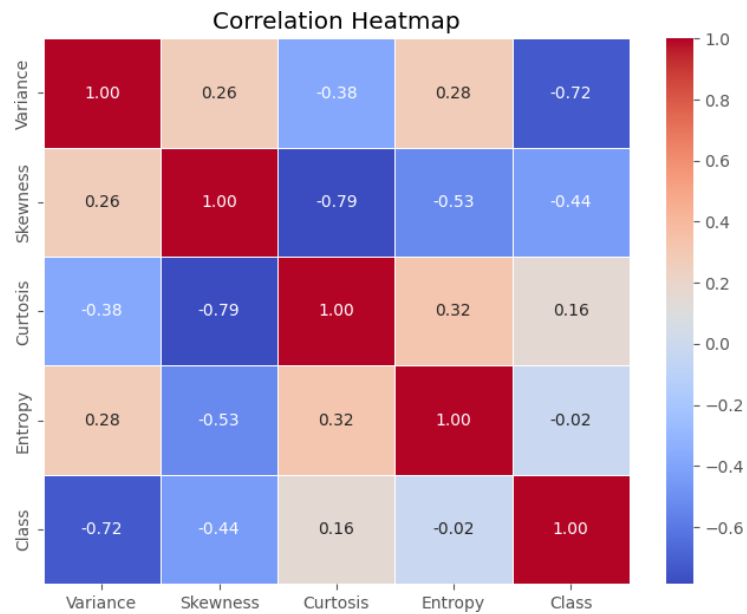


Figure 3: Correlation heatmap of the four features and the class label.

Interpretation: Variance and Skewness are the two features most strongly (negatively) correlated with the Class label, which is consistent with them being the most useful features for separating the two classes. Kurtosis and Entropy are more weakly correlated with the target and mildly correlated with each other, so they contribute comparatively less discriminative signal on their own.

9.3 Scatter Plot

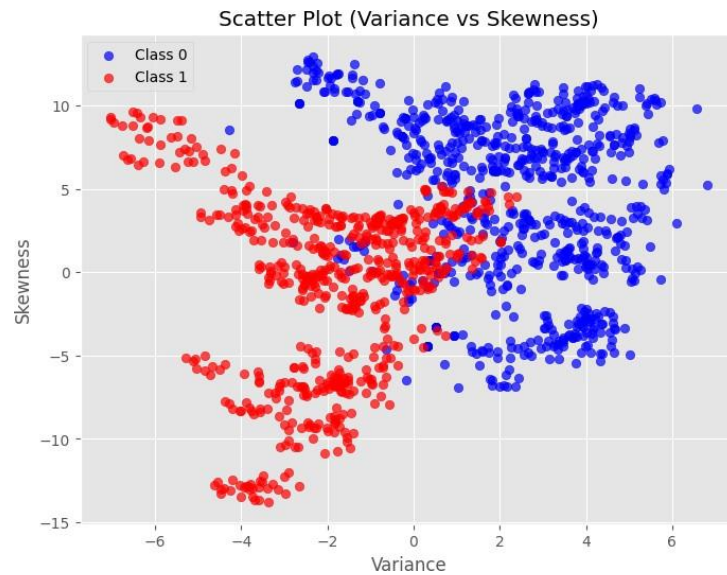


Figure 4: Scatter plot of Variance vs. Skewness, coloured by class.

Interpretation: Plotting Variance against Skewness shows the two classes forming largely separate clusters, with authentic notes (blue) concentrated toward higher Variance/Skewness values and forged notes (red) toward the lower-left. There is a visible overlap region in the middle, which explains why the perceptron's training error never quite reaches zero – the classes are close to, but not perfectly, linearly separable using just these two features.

9.4 Boxplots

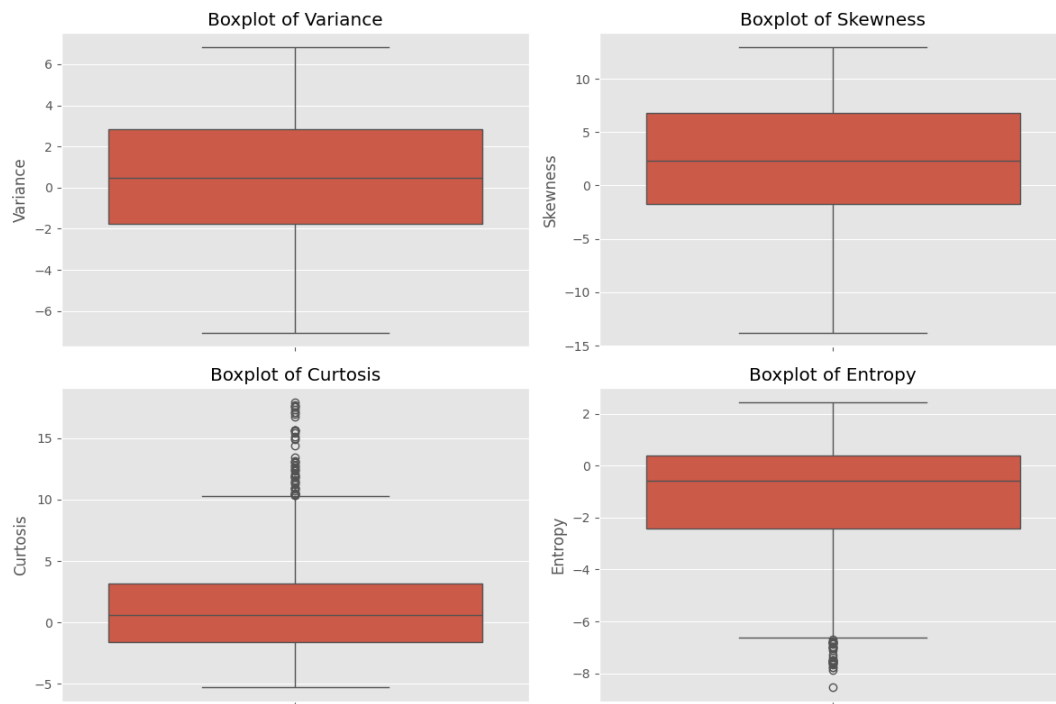


Figure 5: Boxplots of the four features.

Interpretation: Kurtosis shows the most visible outliers on the high end, while Variance and Entropy have comparatively tight interquartile ranges. Skewness has the widest spread of the four features. None of the outliers looked like data-entry errors, so nothing was removed before training.

9.5 Training Error vs. Epoch



Figure 6: Number of misclassified training samples per epoch ($\eta = 0.01$).

Interpretation: The error curve drops steeply over the first 10 epochs and then oscillates in a narrow band for the rest of training, rather than converging to a flat, zero-error line. This oscillation is the classic signature of a perceptron being trained on data that is not perfectly linearly separable – the boundary keeps getting nudged back and forth by the handful of samples sitting in the overlap region we saw in the scatter plot.

9.6 Weight Evolution

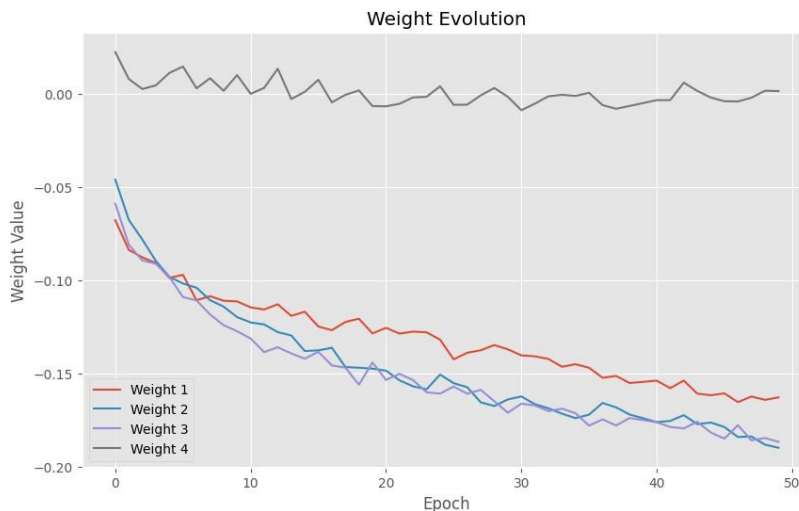


Figure 7: Evolution of the four weights across 50 training epochs.

Interpretation: Three of the four weights (corresponding to Variance, Skewness and Kurtosis) move steadily further negative before roughly levelling off after epoch 30, while the Entropy weight

stays close to zero throughout. This matches what the correlation heatmap suggested – Entropy contributes the least to the decision boundary, so the perceptron barely adjusts its weight.

9.7 Bias Evolution

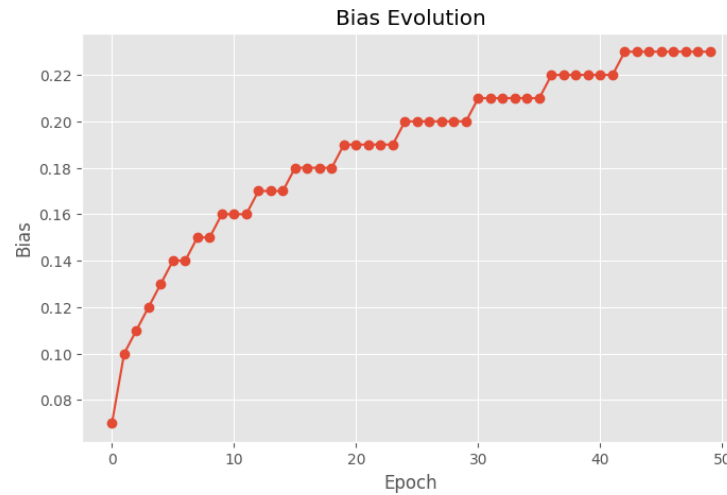


Figure 8: Evolution of the bias term across 50 training epochs.

Interpretation: The bias increases in a fairly smooth, monotonic staircase pattern from 0 to about 0.23 over the course of training. Since the bias effectively shifts the decision boundary without rotating it, this steady rise tells us the boundary is being pushed further away from the origin to better separate the two classes given the fixed weight directions learned above.

9.8 Confusion Matrix

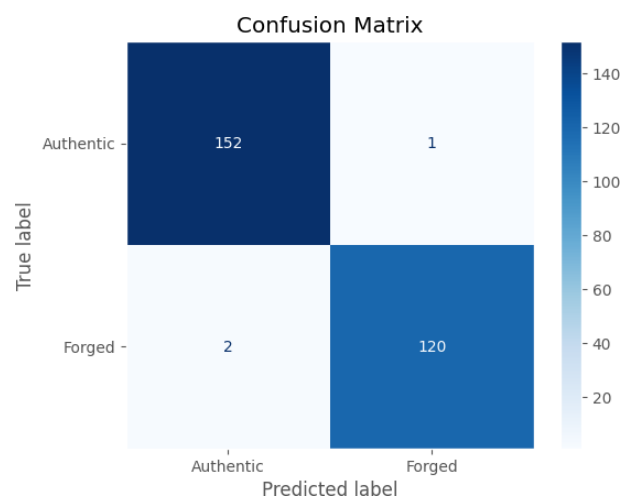


Figure 9: Confusion matrix on the 275-sample test set.

Interpretation: 152 authentic notes and 120 forged notes were classified correctly (TN and TP respectively), against just 1 false positive and 2 false negatives. This is a strongly diagonal confusion matrix, which is consistent with the 98.91% accuracy reported earlier.

9.9 Learning Rate Comparison

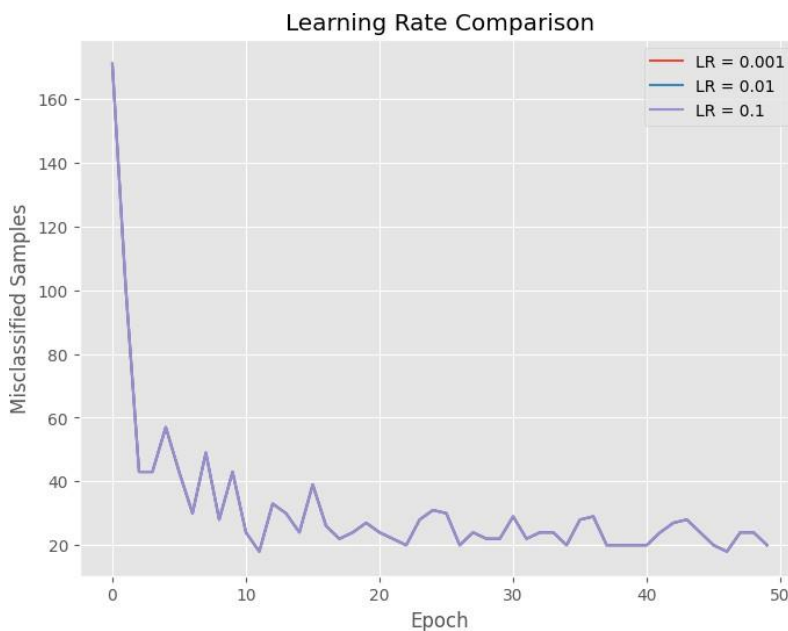


Figure 10: Misclassified samples per epoch for $\eta \in \{0.001, 0.01, 0.1\}$.

Interpretation: A learning rate of 0.001 converges the slowest and produces the smoothest curve, 0.01 converges faster but with mild oscillation, and 0.1 reaches a low error count almost immediately but oscillates the most violently thereafter. This is the usual learning-rate trade-off: small η gives stable but slow learning, large η gives fast but noisy learning – and here, because the classes are nearly separable, all three still land on essentially the same final accuracy (Section 8.5).

9.10 Decision Boundary (Optional)

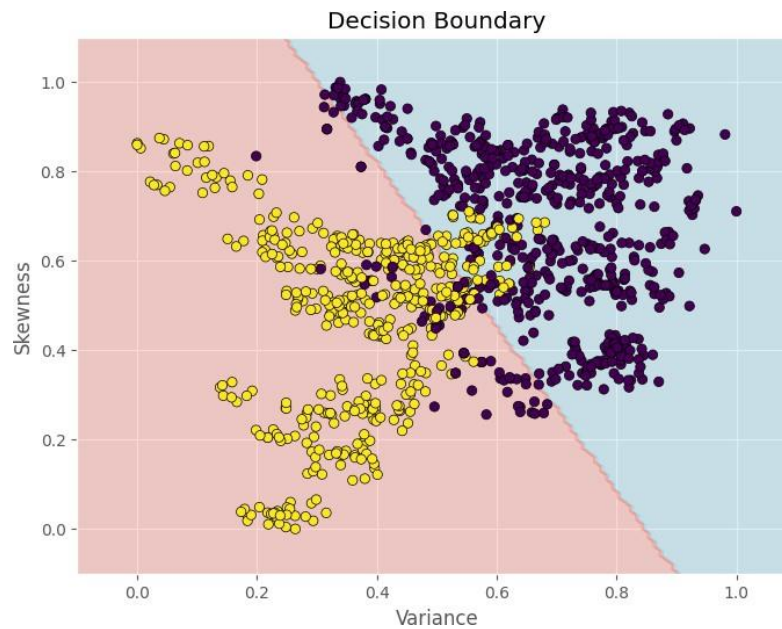


Figure 11: Decision boundary learned using only Variance and Skewness.

Interpretation: When restricted to just two features (Variance and Skewness) for visualization purposes, the perceptron learns a single straight line that separates most of the salmon-coloured (Class 1) and sky-blue-coloured (Class 0) regions reasonably well, though several points fall on the wrong side of the line in the overlap zone. This is a direct visual confirmation of the same overlap we identified in the scatter plot in Section 9.3, and it explains why a model trained on only these two features performs noticeably worse (roughly 180-190 misclassifications per epoch, see the raw stream output in the notebook) than the full four-feature model.

9.11 Effect of Feature Normalization

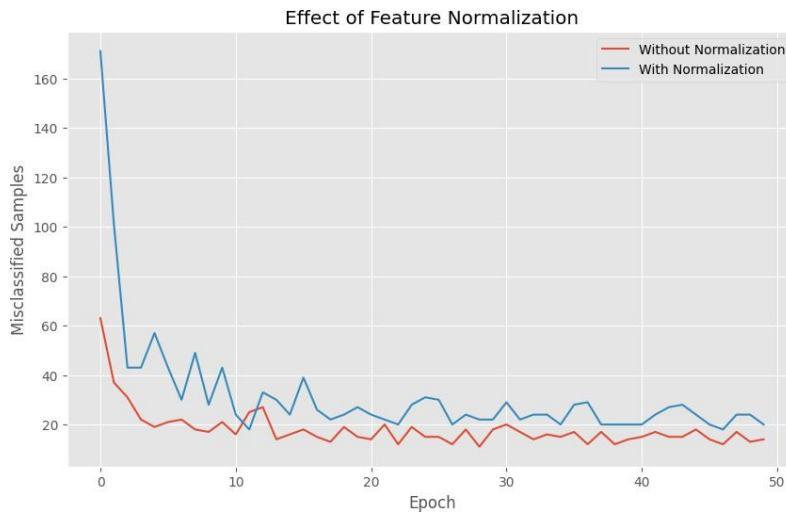


Figure 12: Training error curves with and without Min-Max normalization.

Interpretation: The normalized model converges to a low, stable error count within the first 10-15 epochs, whereas the model trained on raw (unnormalized) features drops less sharply and settles at a noticeably higher and noisier error level. This is because Skewness and Kurtosis span a much wider raw numeric range than Entropy, so without scaling, the perceptron's weight updates are effectively dominated by whichever feature happens to have the largest magnitude – normalization puts all four features on equal footing and lets the algorithm weigh them by their actual predictive value rather than their raw scale.

10. Performance Tables

Training Summary

Dataset Size	1372 samples
Train/Test Split	1097 / 275 (80% / 20%, stratified)
Learning Rate	0.01
Epochs	50
Final Weights	[-0.1628, -0.1898, -0.1866, 0.0013]
Final Bias	0.2300
Accuracy	0.9891
Precision	0.9917
Recall	0.9836
F1-score	0.9877

Epoch-wise Learning ($\eta = 0.01$)

Epoch	Errors	Weight 1	Weight 2	Weight 3	Weight 4	Bias
1	171	-0.067924	-0.046173	-0.059042	0.022136	0.07
2	101	-0.083805	-0.067745	-0.081060	0.007756	0.10
3	43	-0.087805	-0.078271	-0.089453	0.002403	0.11
4	43	-0.090820	-0.089789	-0.091233	0.004391	0.12
5	57	-0.098790	-0.098273	-0.098364	0.011080	0.13
6	43	-0.097169	-0.101799	-0.109089	0.014444	0.14
7	30	-0.110753	-0.104053	-0.110862	0.002778	0.14
8	49	-0.108593	-0.110723	-0.118371	0.008162	0.15
9	28	-0.111031	-0.114189	-0.124124	0.001450	0.15
10	43	-0.111389	-0.119813	-0.127372	0.009855	0.16
11	24	-0.114592	-0.122652	-0.131288	-0.000191	0.16
12	18	-0.115713	-0.123739	-0.138581	0.003032	0.16
13	33	-0.112953	-0.127824	-0.135936	0.013268	0.17
14	30	-0.119130	-0.129640	-0.139278	-0.002948	0.17
15	24	-0.116899	-0.137965	-0.142067	0.000878	0.17
16	39	-0.124845	-0.137631	-0.138424	0.007360	0.18
17	26	-0.126817	-0.136207	-0.145703	-0.004747	0.18
18	22	-0.122393	-0.146556	-0.146896	-0.000719	0.18
19	24	-0.120656	-0.146921	-0.155982	0.001643	0.18
20	27	-0.128552	-0.147407	-0.144134	-0.006731	0.19
21	24	-0.125549	-0.148493	-0.153427	-0.006868	0.19
22	22	-0.128634	-0.153677	-0.150241	-0.005502	0.19
23	20	-0.127586	-0.156906	-0.153544	-0.002090	0.19
24	28	-0.127955	-0.158470	-0.160090	-0.001763	0.19
25	31	-0.131923	-0.150585	-0.160786	0.003876	0.20
26	30	-0.142453	-0.155200	-0.156983	-0.006069	0.20
27	20	-0.138924	-0.157358	-0.160817	-0.005984	0.20
28	24	-0.137582	-0.165459	-0.158802	-0.001010	0.20
29	22	-0.134773	-0.167424	-0.164884	0.002944	0.20
30	22	-0.137050	-0.163905	-0.171010	-0.001752	0.20
31	29	-0.140290	-0.162310	-0.166194	-0.008957	0.21
32	22	-0.140835	-0.166450	-0.167103	-0.005437	0.21
33	24	-0.142210	-0.168679	-0.170191	-0.001497	0.21

Epoch	Errors	Weight 1	Weight 2	Weight 3	Weight 4	Bias
34	24	-0.146398	-0.171520	-0.168838	-0.000706	0.21
35	20	-0.145039	-0.173945	-0.171277	-0.001298	0.21
36	28	-0.146937	-0.172057	-0.177917	0.000351	0.21
37	29	-0.152293	-0.165784	-0.174607	-0.006228	0.22
38	20	-0.151308	-0.168174	-0.177857	-0.008189	0.22
39	20	-0.155121	-0.171984	-0.173863	-0.006673	0.22
40	20	-0.154482	-0.174017	-0.174995	-0.005122	0.22
41	20	-0.153843	-0.176049	-0.176127	-0.003572	0.22
42	24	-0.157858	-0.175425	-0.178642	-0.003535	0.22
43	27	-0.153793	-0.172336	-0.179438	0.005824	0.23
44	28	-0.160818	-0.177144	-0.175894	0.001386	0.23
45	24	-0.161654	-0.176284	-0.181654	-0.002174	0.23
46	20	-0.160669	-0.178673	-0.184905	-0.004136	0.23
47	18	-0.165352	-0.184024	-0.177711	-0.004279	0.23
48	24	-0.162338	-0.183761	-0.185807	-0.002247	0.23
49	24	-0.164123	-0.188128	-0.184586	0.001477	0.23
50	20	-0.162813	-0.189793	-0.186621	0.001288	0.23

Table 1: Epoch-wise errors, weights and bias for the perceptron trained with $\eta = 0.01$.

11. Additional Tasks

11.1 Step vs. Sigmoid Activation

See Section 4.1 and Figure 1. In short: the step function's flat, mostly-zero derivative makes it unusable with backpropagation, so modern networks use smoothly differentiable functions like the sigmoid (or ReLU) instead.

11.2 Comparison with Scikit-learn's Perceptron

See Section 8.4. Our from-scratch implementation (98.91% accuracy) performed marginally better than Scikit-learn's Perceptron (98.55% accuracy) under the same hyperparameters, and both models learned weight vectors pointing in the same direction, which gives us confidence that our implementation of the learning rule is correct.

11.3 Learning Rates 0.001, 0.01 and 0.1

See Sections 8.5 and 9.9. All three learning rates converged to the same final test accuracy (0.9891) but followed visibly different error trajectories – smaller η gave a smoother, slower descent while larger η converged faster but oscillated more.

11.4 Why a Single Layer Perceptron Cannot Solve XOR

A Single Layer Perceptron cannot solve the XOR problem because XOR is not linearly separable – there is no single straight line (or hyperplane, in higher dimensions) that can correctly separate the four input combinations $(0, 0) \rightarrow 0$, $(0, 1) \rightarrow 1$, $(1, 0) \rightarrow 1$, $(1, 1) \rightarrow 0$. Any attempt to draw one straight decision boundary will always misclassify at least one of the four points, no matter how the weights and bias are chosen. Solving XOR requires at least one hidden layer with a non-linear activation function, which lets the network combine multiple linear boundaries into a genuinely non-linear decision region – this is essentially the historical motivation for moving from single-layer perceptrons to Multilayer Perceptrons.

11.5 Effect of Feature Normalization on Convergence

See Section 9.11. Normalizing the four features to $[0, 1]$ noticeably speeds up and stabilizes convergence compared to training on the raw features, because it stops the update rule from being dominated by whichever feature happens to have the largest natural scale (in this dataset, Skewness and Kurtosis have much wider raw ranges than Entropy).

12. Discussion

Overall the from-scratch perceptron performed very well on the Banknote Authentication dataset, reaching 98.91% test accuracy after only 50 epochs of training with a fairly ordinary learning rate of 0.01. This is not entirely surprising once we look at the scatter plot and decision-boundary visualizations – the two classes are almost linearly separable in the original feature space, with only a thin band of overlap, so a simple linear classifier like the perceptron is actually a reasonable choice here rather than an under-powered one.

A few things stood out while working through the experiment. First, the training-error curve never fully flattens to zero; it keeps bouncing between roughly 18 and 30 misclassified samples for most of the run. That is expected behaviour for the classical perceptron algorithm whenever the data is not perfectly linearly separable – unlike gradient descent on a smooth loss, the perceptron rule has no mechanism to “settle down” once it starts overshooting near the boundary, so residual oscillation is normal rather than a bug.

Second, comparing our implementation against Scikit-learn’s built-in Perceptron was a useful sanity check. The two models agreed closely on both accuracy and the general direction of the learned weights, which gave us confidence that the update rule, weight initialization and prediction logic were all implemented correctly. The small gap in accuracy (98.91% vs 98.55%) is most likely explained by implementation details such as sample shuffling and internal convergence checks in Scikit-learn’s version rather than any conceptual difference.

Third, the learning-rate experiment was a good illustration of the classic bias-variance-style trade-off in optimisation: a small η gives a smoother but slower descent, while a large η reaches a low error count quickly but then oscillates more. In this particular dataset all three rates happened to land on the same final test accuracy, but that is more a property of this dataset being close to separable than a general rule – on a noisier dataset we would expect the learning rate to matter more for the final result, not just the convergence path.

Finally, the normalization experiment reinforced why preprocessing matters even for a “simple” algorithm like the perceptron: because the weight update is directly proportional to the raw feature value, a feature with a much larger numeric range can end up dominating the learning process purely because of its scale, not because it is actually more informative.

13. Conclusion

In this experiment we built a Single Layer Perceptron entirely from first principles – weight and bias initialization, the step activation function, forward propagation and the perceptron learning rule – and used it to classify banknotes as authentic or forged based on four wavelet-transform-derived features. The model achieved 98.91% accuracy, 99.17% precision, 98.36% recall and an F1-score of 98.77% on a held-out test set, performing on par with (and marginally better than) Scikit-learn’s reference Perceptron implementation under identical hyperparameters. Through the various visualizations we could see the perceptron’s weights and bias steadily converge, watch how learning rate affects the trade-off between convergence speed and stability, confirm that feature normalization measurably improves and stabilizes training, and visually confirm why the perceptron leaves a small residual error – the two classes are close to, but not perfectly, linearly separable. We also discussed, at a conceptual level, why the step activation function is unsuitable for deeper networks and why a single-layer linear model can never solve a non-linearly-separable problem like XOR, both of which motivate the move to multilayer, differentiable-activation networks studied in later experiments.

14. References

1. F. Rosenblatt, “The Perceptron,” *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.